The Fastest Engine Is Not the Shippable Engine: Replacing a Regex Engine for Data Redaction Under Production Constraints

DANIELE FRISANCO, Independent Researcher, Italy

We report on replacing the regex engine behind a data-redaction library to improve performance under four production constraints—zero runtime dependencies, byte-for-byte-identical output to the incumbent, robustness across all match densities, and thread-safety in a managed runtime—and find, across a sequence of prototypes, that the speed-optimal engine is ruled out by those constraints, and that the pre-filter pipeline usually assumed best for multi-pattern matching does not hold across densities; the best shippable design is a per-pattern lazy DFA with two domain-specific merges.

CCS Concepts: • **Theory of computation** → **Regular languages**; • **Software and its engineering** → **Software performance**; • **Security and privacy** → *Data anonymization and sanitization.*

Additional Key Words and Phrases: regular expressions, multi-pattern matching, data redaction, lazy DFA, performance engineering, Ruby C extensions

In plain terms. This paper is about the search engine inside a data-redaction library: a tool that scans text for secrets and personal data—API keys, bank account numbers, national identifiers—and replaces each with a placeholder before the text is logged, stored, or shared. The library recognizes a fixed, curated catalogue of such token types, not arbitrary user-supplied patterns, and must do so fast, on every line a system produces. We replace that engine, and the central finding is that the *fastest* available engine is not the one a real library can ship: a shippable engine must also add no dependencies, reproduce the previous engine's output exactly, and stay correct under concurrency—each of which rules out a faster option. What follows is the story of finding the fastest engine that meets all three. A glossary of the engines and terms used appears at the end of §2.

1 INTRODUCTION

A data-redaction library scans text for secrets and personal data—API keys, IBANs, national identifiers, emails, IP addresses, private-key blocks—and replaces each with a placeholder. The library this paper is about does so with a set of dozens of patterns, and it began with an embarrassing measurement: its C extension, written to be fast, was several times *slower* than an equivalent loop in the interpreted host language. The obvious reading—that the C was mis-written, or that C does not help here—is a category error. The variable that changed between the two paths was not the language but the regex engine: the C path drove glibc's POSIX `regexec`, the interpreted path drove the host's own engine with a Boyer–Moore literal pre-filter and bounded per-call state. The slow result is about the engine, not the language; it only looks like a language problem (§5.1).

That observation reframes the project. The goal is performance, but performance is not the only thing we are allowed to optimize. A shippable redaction engine must also (i) add no runtime dependency, (ii) produce byte-for-byte-identical output to the engine it replaces, and (iii) be safe under a threaded, garbage-collected runtime—and it must be fast not on a representative payload but across the whole range of match densities, because a redaction library cannot choose its input. Each of those constraints disqualifies an option that is attractive on raw speed: the fastest engine we measured fails the dependency constraint, the textbook single-automaton speedup fails the exactness constraint, and the fastest prototypes failed thread-safety until a specific refactor. That is

Author's address: Daniele Frisanco, Independent Researcher, Italy, daniele.frisanco@gmail.com.

the paper's thesis: *the fastest engine is not the shippable engine*, and finding the shippable one is a constrained search, not a benchmark.

We ran that search as a sequence of prototypes, each isolating one decision, and report it honestly—including the dead ends, because the search is the evidence. Two findings are sharper than we expected. First, the conventional wisdom for multi-pattern matching—put a shared literal pre-filter in front of the engine—does not survive the density axis: in our configuration the pre-filter pipeline is slower than the same engine without it at *every* density, and its penalty grows monotonically as matches become common, until the hand-built C pipeline is slower than the interpreted loop it was meant to replace (§5.2). Second, the shippable design is not the fast single automaton but a *per-pattern* lazy DFA with two domain-specific merges, whose cost stays flat across densities precisely because it avoids the pre-filter trap (§5.3).

Contributions.

- A constraint framing for engine replacement under production conditions: performance as an objective subject to three binary gates (zero-dependency, exact-output, thread-safe) and a worst-case-over-density objective, with each gate tied to the attractive option it eliminates (§3).
- A density-resolved measurement showing the shared-literal pre-filter pipeline is dominated by the same engine without it across the sampled range, with the mechanism stated generally and the magnitude scoped to our configuration (§5.2).
- A shippable, zero-dependency engine—a per-pattern lazy DFA plus two exactly-disjoint selective merges—that is flat across densities and beats the host engine on every payload (§5.3).
- A reusable systems result: how to make a multi-pattern lazy-DFA matcher re-entrant in a managed runtime by splitting state into shared-immutable and per-thread halves, *including the lazy cache the construction fills mid-scan* (§5.4).

2 BACKGROUND

The task. Redaction is multi-pattern search-and-replace over untrusted text. Each pattern recognizes one class of sensitive token; a scan must find the matches of every pattern and replace them. What counts as the right *set* of matches when two patterns overlap is itself a design decision, and one we revisited mid-project (§7): the original behavior was whatever a loop of per-pattern `String#gsub` calls produced, and we later changed it deliberately. The pattern set here numbers in the dozens and spans long-prefixed credentials (`AKIA. . .`, `sk_live_. . .`, PEM key headers), structured tokens (IBANs, credit-card numbers, emails, IP addresses), and bare fixed-length digit strings (national identifiers). The mix matters: some patterns carry a long literal anchor that a pre-filter can exploit, and some are almost pure digits with no distinctive literal at all, which is the class that defeats literal pre-filtering and shapes the rest of the paper.

The baseline and the incumbent. Two implementations existed before this work. The reference is a pure-Ruby loop that applies each pattern with `String#gsub` in turn; we treat it as the oracle for Gate 2, with the caveat—developed in §7—that this sequential loop has its own overlap semantics (each pattern rewrites the regions it matches, so the earlier-listed pattern wins a contested region) which we initially reproduced exactly and later chose to change. The incumbent is a C extension intended to be faster, which calls glibc `regexec` on each pattern in turn. The C extension also carried two prior optimizations—a literal pre-check before each `regexec` and input chunking to bound the per-call allocation—which together helped but left it still several times slower than the pure-Ruby loop. Those optimizations treated symptoms; the root cause is the engine.

Why a C extension was expected to win, and why it did not. The expectation that native code beats the interpreter is reasonable in general and wrong here, because the comparison is not C versus the interpreter but glibc `regexec` versus the host's Onigmo engine. Onigmo applies a Boyer–Moore literal scan [2] before NFA evaluation, skipping positions that cannot match, and evaluates against a fixed-size stack; glibc `regexec` evaluates the automaton at every position and allocates state proportional to the input on each call. On text where most positions hold no secret, those two differences favor the interpreted path decisively. The redaction engine's job is therefore to combine a fast multi-pattern filter with an exact confirmation step that does not pay either of glibc's costs—which is the design space the prototype search explores (§4).

Glossary. The terms and engines used throughout:

Redaction Replacing each sensitive token in text with a placeholder.
Pattern A rule recognizing one class of sensitive token (e.g. an IBAN).
Match density Sensitive tokens per KB of text; the input axis the library cannot control.
IBAN International Bank Account Number; a structured token type redacted here.
PEM The text wrapper around private-key blocks (`-----BEGIN...`).
NFA / DFA Nondeterministic / deterministic finite automaton—the two standard regex execution models; a DFA has one active state at a time, an NFA may track many.
Lazy DFA A DFA whose states are built on demand during the scan rather than all up front.
Aho–Corasick A classic algorithm that scans for many fixed strings at once in a single pass; used here as a literal pre-filter.
Boyer–Moore A single-string search algorithm that skips ahead on mismatch; the host engine's literal pre-filter.
Literal pre-filter A fast scan for fixed substrings that skips positions which cannot match, fronting a slower engine.
glibc `regexec` The POSIX C-library regex function; the incumbent engine being replaced.
Onigmo The regex engine inside Ruby; the interpreted baseline.
PCRE2 Perl-Compatible Regular Expressions (v2), a widely used regex library with a JIT backend.
RE2 Google's automaton-based regex library (referenced for its multi-pattern `Set` state-explosion limit).
JIT Just-in-time compilation; PCRE2's JIT compiles patterns to native code, the fastest engine measured but dependency-disqualified.
GVL Ruby's Global VM Lock; releasing it lets other threads run during a C call, but only if the C code is re-entrant.
Oracle / exact output (Gate 2) The previous engine's output, which the replacement must reproduce byte-for-byte.
Selective merge Combining a few patterns into one pass only where their structure makes it exactly safe.

3 THE CONSTRAINT BOX

The objective is performance, but it is performance constrained, and the constraints are what make the problem non-trivial—each one disqualifies an option that is otherwise attractive on speed alone. We treat three of them as binary gates: a candidate either passes or is ineligible, regardless of how fast it is. The fourth is the shape of the objective itself.

Gate 1: zero runtime dependencies. The library installs as a gem with no native dependencies beyond what Ruby itself links. This rules out the speed-optimal engine outright. Plain PCRE2 JIT is

the fastest engine we measured at every density (§6), and it is disqualified not for being slow but for requiring a third-party shared library at run time. The gate converts the fastest option into a non-option, which is the paper's thesis in miniature.

Gate 2: byte-for-byte-identical output. The replacement must redact exactly the same byte ranges as the incumbent, character for character, so that swapping the engine is invisible to every downstream consumer. This is the gate that kills the most tempting algorithmic shortcut. Merging all patterns into a single combined automaton—the textbook way to make multi-pattern matching one pass—was among the fastest prototypes we built (§4), but it does not preserve per-pattern semantics: a state in the merged automaton is a set of NFA states drawn from all patterns at once, so one pattern's accept condition can be reached through state shared with another, producing matches the per-pattern incumbent never makes. On our correctness suite the merged design passed only 4 of 13 cases (31%), and it cannot be patched without per-pattern submatch tracking—which removes the very sharing that made it fast. We enforce the gate with an oracle that compares the candidate's output to the pure-Ruby `gsub` reference byte-for-byte over our input corpus (§4); a single mismatch fails the candidate.

Gate 3: thread-safety in a managed runtime. The engine runs inside a garbage-collected host with multiple threads and a global interpreter lock that we want to release on large inputs (to let other threads run during a long scan). Several fast prototypes kept mutable matcher state in process globals—including, subtly, a lazy DFA cache that is filled *during* a scan—which is a data race the moment two threads scan at once or the lock is released mid-scan. The gate is what forces the immutable/mutable state split described in §5.4; until that refactor, the otherwise-shippable engine was ineligible.

The objective: robustness across all match densities. Among the candidates that pass all three gates, we do not optimize average-case throughput on a representative payload. We optimize the worst case across match density, because a redaction library cannot choose its input: the same code path must handle a log line with one secret in ten kilobytes and a credential dump that is almost entirely secrets. An engine that is fastest on typical text but degrades on dense input fails the use case even though it would win a benchmark average. This objective is why the density sweep (§5.2) is the decisive measurement rather than a single headline number, and why a design whose cost is flat across densities is preferred over a faster-on-average one that is not.

4 METHOD: THE PROTOTYPE SEARCH

We did not arrive at the shippable engine by design; we arrived at it by a search through roughly two dozen prototypes, each a small C matcher measured against the same oracle and the same payloads. The method is the contribution as much as the result: every prototype is a falsifiable claim about what makes the workload fast, and several attractive ones were killed by the gates rather than by speed.

The prototype ladder. The search spans engines that swap one variable at a time. The early rungs front a shared Aho–Corasick filter onto different confirmation engines—glibc `regexec`, Onigmo, PCRE2 (interpreter and JIT)—which is how we separated the engine effect from the language effect (§5.1) and built the pre-filter pipeline whose density penalty we report (§5.2). A separate line abandons the shared filter for per-pattern automata: a merged Thompson NFA over all patterns (fast but only 31% correct—Gate 2), then a per-pattern Thompson bytecode VM, then literal and first-byte pre-filters on it, then the per-pattern lazy DFA (§5.3) and its two selective merges. We report the null results too—a cross-pattern union bitmap and a precomputed-initialization variant

that nibbled at non-bottleneck costs and did not move the hot path—because the discarded branches are evidence about where the cost actually is.

The exact-output oracle. Gate 2 is enforced mechanically. The reference is the pure-Ruby `gsub` loop over the pattern set; for a candidate to be eligible, its output must be byte-for-byte identical to the reference on every input in our corpus, and we compare the full set of (pattern, start, length) matches, not just the redacted string, so that a coincidentally equal output cannot hide a wrong match. The corpus includes the four payloads below, pure-noise text, targeted edge cases for each merge (buffer boundaries, back-to-back matches, near-miss prefixes), and a several-hundred-scan sequential stress run that catches cross-call state bugs. A single mismatch fails the candidate; the shippable engine passes the oracle outright.

Payloads and the density axis. We measure four fixed payloads—sparse, medium, dense, and an all-secrets "env" payload—and, for the crossover result, a continuous sweep that varies match density along a single construction. The sweep holds the hit mix, the noise text, the random seed, and the ~ 1 MB buffer size fixed and varies only the spacing between hits, so the four fixed payloads become points on one continuous curve rather than a separate measurement regime. Densities are log-spaced from roughly 0.1 to 38 hits per kilobyte.

Timing and environment. Each engine is initialized once per configuration, given one warm-up scan, then timed over repeated reps; the headline metric is the minimum per-rep time, which is the least contaminated by scheduler and GC noise, with median and mean recorded alongside to show spread. All engines run in one process under identical conditions, so the *shape* of every comparison is robust even where absolute magnitudes carry a caveat (§7). The full hardware, OS, compiler, and library versions are recorded with the data; the incumbent we compare against is a faithful reproduction of the pre-replacement glibc engine, discussed in §7.

5 FINDINGS

5.1 The incumbent is slow for engine reasons, not for being C

The starting point is a result that looks, at first, like an indictment of the implementation language: the C extension is 3–5× *slower* than an equivalent pure-Ruby loop that calls `String#gsub` with the same pattern set. A C extension is normally expected to beat the interpreter it extends, so this inversion is the paper's entry point. We argue it is a category error to read it as "C is slower than Ruby here." The controlled variable is the regex engine, not the language: the C path drives glibc's POSIX `regexec`, while the Ruby path drives Onigmo, Ruby's own engine. The two engines differ in two ways that both favour Onigmo on this workload.

First, a pre-filter gap. Onigmo applies a Boyer–Moore literal scan before committing to NFA evaluation: at a position with no required anchor byte it skips ahead by the shift table. Many redaction patterns carry long literal prefixes (`sk_live_`, `AKIA`, `--BEGIN`), so on ordinary text Onigmo rejects most positions cheaply, whereas glibc `regexec` evaluates the automaton at every position. Second, a per-call setup cost: glibc `regexec` rebuilds an internal search structure over the input on each call—work proportional to the remaining input length—whereas Onigmo evaluates against a fixed-size stack; with one pass per pattern, that per-call cost is paid once per pattern per input. Neither factor is about C.

To see where the instructions actually go, we profiled the three engines under Callgrind on one shared payload and bucketed each function's instruction count into mechanisms (Table 1).[1]

[1]Callgrind counts instructions retired per function exactly rather than sampling cycles, so the attribution is deterministic and reproducible across runs and machines; we report it in place of `perf` cycle samples, which hardware-counter access policy on our machine disallowed. Instruction counts are not wall-clock cycles, and we read the table qualitatively. Onigmo's

Table 1. Where the instructions go: Callgrind instruction-count attribution (% of instructions retired) on one shared ≈1 MB payload, by mechanism. glibc's cost is automaton evaluation plus $O(N)$ per-call setup, not allocation. (†) Onigmo's hot path is stripped library code we cannot split into pre-filter versus automaton, so it is reported as a single bucket.

engine	automaton eval.	per-call setup	literal pre-filter	allocation	match copy-out	other
glibc `regexec`	73.4	25.9	<0.1	0.5	0.1	0.2
Onigmo	96.7†	<0.1	—†	0.6	0.3	2.4
v19 lazy DFA	91.3	<0.1	6.0	0.1	<0.1	2.6

The profile sharpens—and partly corrects—the account above. For glibc, the cost is dominated by automaton evaluation (≈ 73%) plus the per-call search-string setup (≈ 26%); *heap allocation is negligible* (< 1% of instructions in our profile, corroborated by an independent timing split that put `malloc`/`free` churn near zero and `regexec` itself at ≈ 94%). The incumbent is slow not because it allocates, but because it runs a full automaton at every position with no literal pre-filter to skip the overwhelmingly non-matching ones, and pays an $O(N)$ setup on every per-pattern pass. The shipped v19 engine, by contrast, spends its instructions in the lazy-DFA transition stepping it was designed around, with a small literal pre-filter pass (`memmem`, ≈ 6%) and no measurable per-call allocation—which is the mechanism the rest of this section builds on.

These two effects are not independent of density, which sets up the rest of the paper: the pre-filter advantage is largest exactly when hits are rare (most positions can be skipped) and shrinks as hits become common. Figure 1 places the reproduced pre-v19 glibc engine (§4) against the field across the density range; in our configuration it sits roughly an order of magnitude above every shippable engine, including the pure-Ruby loop it was meant to replace.

5.2 The pre-filter pipeline penalty grows with match density

The conventional move for multi-pattern matching is to put a shared literal pre-filter in front of a confirmation engine—an Aho–Corasick or Boyer–Moore pass that finds candidate positions, followed by a full regex check only there. We built that pipeline (Aho–Corasick + Boyer–Moore + PCRE2 JIT) and measured it against the same PCRE2 JIT engine with no pre-filter, across a continuous sweep of match densities (Fig. 2, Table 2).

The mechanism is general: a literal pre-filter pays off only by *rejecting* positions before the expensive engine runs. Its benefit is therefore bounded by how many positions it can reject, and that fraction falls as matches become denser. Past some density the pre-filter inspects nearly every position, finds a candidate at most of them, and so adds its own cost on top of a confirmation step it can no longer avoid. The pre-filter becomes pure overhead.

The magnitude, in our configuration, is sharper than the mechanism alone predicts. The pipeline is not merely worse at high density—it is slower than the same JIT without the pipeline at *every* density we sampled, from the sparsest (≈ 0.1 hits/KB) to the densest (≈ 38 hits/KB). The penalty grows monotonically with density, from 1.2× slower to 11× slower. Onigmo, which also fronts its automaton with a Boyer–Moore scan, rises with density on the same payloads—independent corroboration that the effect is a property of pre-filtering, not of one implementation.

The single sentence we take to §6 is the crossover against the incumbent the pipeline was meant to beat: *the optimized C pipeline overtakes the pure-Ruby* `gsub` *loop at* ≈ 12 *hits/KB*—below that density it wins, above it the hand-built pipeline is slower than the Ruby it was built to replace.

hot functions are stripped library internals we cannot split into pre-filter versus automaton, so its share is reported as one bucket (†). Generated by `prototypes/.../profile_engines.sh` and `paper/gen_profile_table.py`.

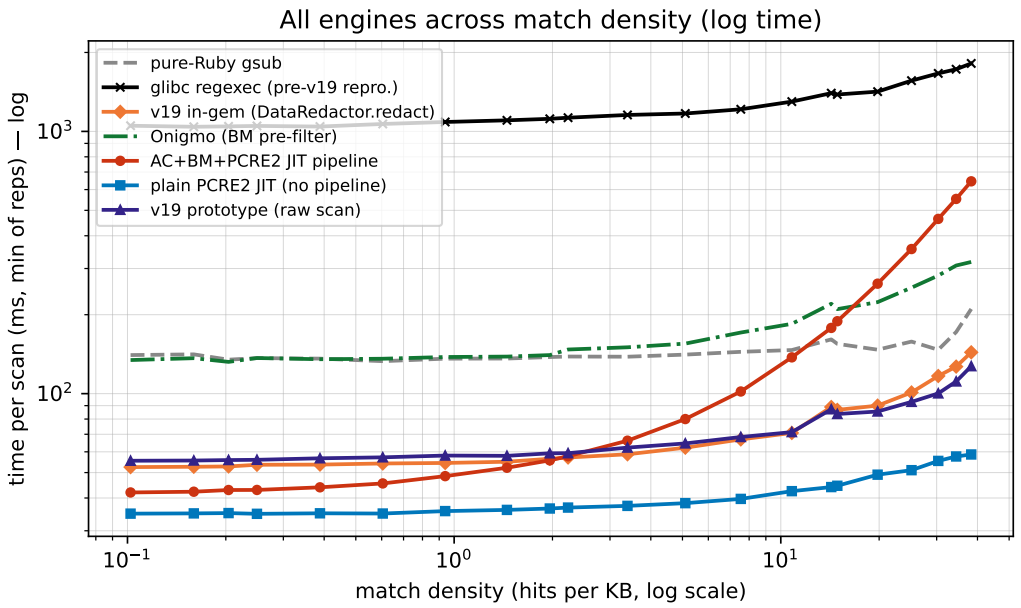

Fig. 1. Time per scan vs. match density (log–log), all engines. The reproduced pre-v19 glibc `regexec` engine sits roughly an order of magnitude above every shippable engine—slower than the pure-Ruby gsub loop it was meant to beat. All engines, including the glibc baseline, are measured at the same operating point (idle machine, performance clock; see §7 and `environment.md`).

Table 2. Time per scan (ms, min of reps) at selected densities. All engines, including the glibc baseline, measured at the same operating point (idle machine, performance clock; §7). Generated by `paper/plot_density_sweep.py`.

hits/KB	pure-Ruby gsub	glibc regexec	v19 in-gem	Onigmo	AC+BM+PCRE2 JIT pipeline	plain PCRE2 JIT	v19 prototype
38.28	210.3	1816.5	143.7	317.8	645.1	58.6	127.2
14.27	161.0	1399.1	88.9	220.2	178.1	44.0	87.2
1.96	137.9	1116.9	56.4	140.3	55.7	36.5	59.2
0.20	134.8	1045.2	52.8	132.2	42.9	35.0	55.8
0.10	140.3	1051.1	52.5	134.3	42.0	34.9	55.5

Plain PCRE2 JIT and the v19 lazy-DFA engine (§5.3) show no such climb; they stay flat across the whole range, which is what a design selected for worst-case density robustness must do.

5.3 The shippable design: per-pattern lazy DFA + two selective merges

The engine that passes all three gates and is flat across densities is built from one established technique and two domain-specific specializations.

Per-pattern lazy DFA.. The base is Cox's lazy (hybrid) DFA construction [3]: build DFA states on demand from the NFA and cache the hot ones, so the per-byte inner loop collapses from work proportional to the live NFA-state set into a single table lookup, `state = trans[state][byte]`. Our one deliberate departure is to keep *one small DFA per pattern* rather than a single merged automaton over all patterns. That choice is what makes the lazy DFA usable here at all: a merged automaton is

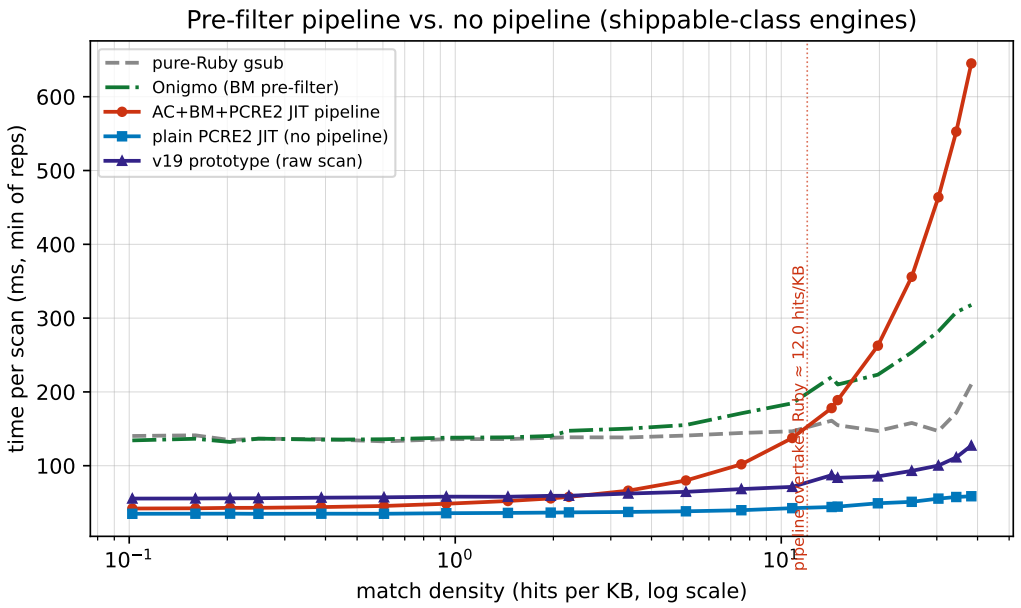

Fig. 2. Time per scan vs. match density (shippable-class engines, linear y; clean run). The AC+BM+PCRE2-JIT pipeline tracks the no-pipeline engines when hits are rare, then its cost rises with density and overtakes the pure-Ruby baseline at ≈ 12 hits/KB. Plain PCRE2 JIT and the v19 lazy-DFA engine stay flat. Relative to plain PCRE2 JIT, the pipeline ranges from 1.2× slower (sparsest) to 11× slower (densest) in our configuration.

exactly the design that failed Gate 2 (§3) and the design whose state set explodes past a few dozen patterns in RE2::Set and Rust's `RegexSet` [4, 5]. Per pattern, each DFA is tiny—most of ours settle at one to a few dozen states—so there is no explosion and no cross-pattern contamination, while each automaton is still byte-for-byte faithful to the incumbent. States that cannot be encoded as a DFA position (line-anchored patterns) fall back to the exact NFA inner loop, so correctness never depends on the optimization firing.

Two selective merges. On top of the per-pattern engine we merge two subsets of patterns that the credential/PII domain happens to contain in exactly-disjoint form. We frame these as *specialization*, not as a general rule-grouping algorithm (§8): each is recognized by a syntactic test, not learned by partitioning a conflict graph.

- *Pure-digit group.* Nine patterns whose entire body is `[0-9]{`lo`,`hi`}` (national identifiers of various fixed lengths) are replaced by one linear pass that finds each maximal run of digits and, for a run of length L, emits exactly the members whose length window contains L. Every member is boundary-wrapped—each pattern is bracketed by a non-alphanumeric boundary, (^|[^0-9A-Za-z])(...)([^0-9A-Za-z]|$), so a digit run only matches when flanked by a non-digit or a buffer/line edge. That is why a shorter member can never match a sub-segment of a longer digit string: the interior of a longer run has digits on both sides and fails the boundary. Working on *maximal* runs therefore reproduces the incumbent's per-pattern non-overlapping match stream byte-for-byte, including the line-edge cases—rather than nine independent scans of the buffer.

- *IBAN union pass.* The eighteen IBAN patterns each begin with a unique two-letter country code; in the per-pattern engine each runs its own whole-buffer literal sweep for that code—eighteen sweeps that, on text without those letter pairs, find nothing yet still cost a flat fraction of every scan (we measured ∼ 11% of full-scan time, present or not). One pass replaces them: a two-byte prefix table maps each country code to its single owning pattern (codes are unique, so the map is 1 : 1), and only the matched pattern's DFA is driven at each candidate. This is dispatch-by-distinguishing-literal, the same shape as the Aho–Corasick prefix filter, specialized to a direct table.

Both merges are exact: an oracle run confirms byte-for-byte-identical output to the incumbent on the full corpus plus targeted edge payloads, so neither weakens Gate 2. Their effect is to remove redundant whole-buffer work that the per-pattern engine pays regardless of input, which is why they help most on sparse text—the common case—without hurting dense text.

Result. In our configuration the resulting engine ("v19") is the fastest *zero-dependency* design we built: it beats Onigmo on every payload and stays flat across the density range where the pre-filter pipeline climbs (Fig. 2). It does not beat plain PCRE2 JIT—nothing zero-dependency does—but PCRE2 JIT is ineligible (Gate 1), which is the whole point.

5.4 Prototype-to-production hardening: GVL-free re-entrancy

The engine above passes the speed and exactness gates, but the prototype kept its per-scan state—cursors, a generation stamp, and the warm lazy-DFA cache—in process globals. That is fine in a single-threaded harness and a data race the moment two threads redact at once, which is the normal case in a threaded Ruby server. Passing Gate 3 required a refactor that is the paper's reusable systems result, separable from the algorithm.

Immutable/mutable split. We partition the engine state by sharing rule. The build-once data—the compiled program, length bounds, the first-byte filter, literal-skip hints, and merge membership—is immutable after construction and shared read-only across threads. Everything written during a scan—the NFA scratch, the merge cursors, the generation stamp, *and the lazy-DFA cache itself*—moves to per-thread storage. Putting the *cache* on the per-thread side, not just the scratch, is the non-obvious move: it means the cache warms independently per thread and is never written by two threads at once. A generation counter, bumped when the pattern set changes, makes a thread rebuild its cache lazily rather than requiring cross-thread invalidation.

Releasing the lock, bounding the cost. Because a scan now performs no shared writes, the redaction call can release the interpreter lock around the built-in pass for inputs above a few kilobytes, so a large redaction on one thread no longer stalls the others; small inputs keep the lock to avoid the release overhead. The price is one DFA cache per scanning thread. Two details keep that affordable. First, an allocation floor: the per-pattern state array starts at eight states and doubles, and since most patterns settle at a handful (1–14 states; 45 at most), the steady-state footprint is ≈ 0.86 MB per scanning thread—down from ≈ 3.2 MB before the floor was tuned—with no throughput change, since the few larger DFAs just do a couple of extra warm-up doublings off the hot path. Second, reclamation: a thread-exit destructor frees the whole per-thread block, so a process that churns short-lived scanning threads keeps flat memory instead of leaking a cache per dead thread, while fixed thread pools simply reuse the block.

Generality. The result is that a multi-pattern lazy-DFA matcher can be made re-entrant and lock-free-to-release by partitioning state into build-once-shared and per-thread halves—*including the lazy cache that the construction fills during a scan*—with the per-thread cost bounded by an

Table 3. Eligibility under the three gates. "Fast" is relative standing on the density sweep, not an eligibility criterion.

Engine	Zero-dep	Exact	Thread-safe	Fast
Plain PCRE2 JIT	no	yes	–	fastest
Merged automaton	yes	*no*	–	fast
AC+BM+JIT pipeline	no	yes	–	density-dependent
Pure-Ruby gsub	yes	yes	yes	slow–medium
v19 (this work)	yes	yes	yes	flat, fast

allocation floor and a thread-exit reclaim. This is the kind of constraint (thread-safety in a managed runtime) that does not appear in an algorithm micro-benchmark and yet decides whether the algorithm is shippable.

6 EVALUATION

The evaluation answers two questions in order: which engines are *eligible*, and among those, which is fast where it counts.

Eligibility. Table 3 summarizes the gates. Plain PCRE2 JIT is the fastest engine at every density and is ineligible: it needs a third-party library at run time (Gate 1). The merged single automaton is fast and ineligible: it diverges from the incumbent (Gate 2, 4/13 correct). The per-pattern lazy DFA with selective merges is eligible on all three gates—it adds no dependency, passes the byte-for-byte oracle, and is re-entrant after the state-split refactor (§5.4). The pre-filter pipeline is eligible but loses on the objective, below.

Performance where it counts. Among eligible designs, the deciding measurement is the density sweep (Fig. 2, Table 2), because the objective is worst-case-over-density, not average throughput. Two results carry the section. First, the pre-filter pipeline is dominated by the same JIT without the pre-filter at every sampled density, with the penalty growing from 1.2× to 11× as density rises, and it crosses the pure-Ruby baseline at ≈ 12 hits/KB—so the elaborate pipeline is, past that point, slower than the loop it was built to replace (§5.2). Second, v19 stays flat across the entire density range and beats the host's Onigmo engine on every fixed payload, while remaining within measurement noise of its own raw-scan prototype once the gem's string-allocation overhead is accounted for. It does not beat plain PCRE2 JIT—no zero-dependency engine does—but plain PCRE2 JIT is ineligible, which is exactly the paper's point: the engine we ship is the fastest *eligible* one, and its flatness across densities is what the objective demanded.

7 DISCUSSION AND THREATS TO VALIDITY

Mechanism versus magnitude. We separate two kinds of claim. The mechanisms—that a literal pre-filter's benefit is bounded by the positions it can reject and so erodes with density; that a merged automaton cannot preserve per-pattern semantics without reintroducing the work the merge removed; that per-thread state including the lazy cache makes the matcher re-entrant—are properties of the designs and do not depend on our hardware. The magnitudes—a 1.2× to 11× pipeline penalty, a crossover near 12 hits/KB, the absolute milliseconds—are ours, measured on one machine with one pattern set and one corpus, and we scope every such number accordingly. A reader may reasonably expect a different crossover density on a different pattern mix; the direction of the effect is what we claim generally.

From prototype bench to shipped path. A standing risk in prototype-driven work is that the number that justified the design is measured in a harness the production system never executes. We checked this directly. The headline prototype speedups of the chosen engine over the host's Onigmo, measured in the standalone C harness against the fixed pattern table, are 2.21×/2.16×/1.80×/1.84× for the sparse, medium, dense, and environment-variable payloads. Re-measured *in the gem*, through the real `DataRedactor.redact` entry point after the port—string in, redacted string out, with the library's own allocation and pattern-resolution work on the path—the same payloads give 2.33×/2.30×/1.75×/1.82×. The two sets agree within measurement noise: the matcher is the same matcher, and the small shifts—tenths of a multiplier, in both directions—are the gem's per-call string allocation and pattern-resolution overhead, not a change in the engine. The point is not the second decimal place but that the prototype's verdict held when the engine moved onto the shipped path, where it is the number that actually reaches users.

The exact-output target changed mid-project, on purpose. Gate 2 demands byte-for-byte-identical output to a reference, but we revised the reference once, and the revision is worth stating plainly because it cuts against a naive reading of "exactness." We *first* held the new engine to the original behavior exactly: reproduce, byte-for-byte, what the sequential loop of per-pattern `String#gsub` calls produced, including its overlap rule—when two patterns can claim an overlapping region, the earlier-listed pattern rewrites it first and wins. That semantic was never a designed choice; it was an accidental by-product of rewriting one pattern at a time. We *then* found it could leave a secret partly unredacted: when a long secret's prefix is itself a shorter pattern, the shorter match consumes the region and the trailing bytes of the real secret survive (an access key whose first segment matches a generic token, for example). So in a second step we deliberately changed the contract to longest-match-wins, with equal-length ties broken by pattern order, which redacts the full span and aligns with the overlap semantics of mainstream engines. The lesson for the constraint framing is that "identical output" is the right gate only once the reference itself is right; reproducing a reference exactly and then correcting the reference are two different commitments, and an exactness gate should not freeze a latent defect in place. The benchmarks in this paper were taken against the exact-reproduction phase, so they are unaffected: the resolver change is a semantic one, and we measured no throughput change from it.

The reproduced baseline. The incumbent we benchmark is a faithful reproduction of the pre-replacement glibc engine (plain sequential `regexec`, no shared filter, boundary-wrapped, scan-only), not the original shipping code. The original is gone: the library has since been improved and now ships the engine this paper describes, so a side-by-side run against the literal old code is no longer possible. We verified the reproduction finds the same matches as the current engine on a multi-pattern sample, and we state plainly that the baseline is a reproduction so that no reader mistakes it for the historical binary.

One operating point. All engines, including the slow glibc baseline, are timed on the same otherwise-idle machine at the performance clock (cores boosting to ≈ 4.7 GHz under load on this `amd-pstate` system; see `environment.md`). An earlier draft measured the baseline under a power-saving governor and was kept only as a second operating point; we re-ran the baseline at the performance clock so the headline table is a single consistent comparison rather than a spliced one, and that draft is retained in the data record for provenance. As a robustness aside, the engine ordering and the crossover were identical under the power-saving draft—the conclusions never depended on the clock, only the absolute milliseconds did.

Timing methodology. We report the minimum per-rep time as the headline, which captures the least-contaminated reps and discards downclocked or scheduler-disturbed ones; median and mean

are recorded to show spread. All engines run in one process under identical conditions, so the relative shape of every comparison is robust even where absolute magnitudes carry the caveats above. The sweep is single-threaded; the re-entrancy result (§5.4) is a separate, correctness-oriented claim about parallel safety, not a parallel-throughput measurement, and we do not claim a speedup from releasing the lock—only that other threads are no longer blocked during a large scan.

Scope of the engine. The selective merges exploit structure specific to this domain (exactly-disjoint pure-digit and unique-prefix subsets) under a restricted pattern contract (no backreferences, lookaround, captures, or Unicode classes). They are specializations, not a general grouping method; on a pattern set without such disjoint subsets they simply do not fire, leaving the per-pattern lazy DFA, which is itself general within the contract.

8 RELATED WORK

Automata used as-is. The per-pattern engine is a direct application of the lazy (hybrid) DFA [3]: build DFA states on demand and cache the hot ones, the standard way to make NFA simulation fast without precomputing a full DFA. The shared prefix filter on the early prototypes is Aho–Corasick [1], and the literal skip that the host engine uses—and that the pre-filter pipeline reuses—is Boyer–Moore [2]. We claim no novelty in these; our use of them is conventional.

Multi-pattern matchers and the state-explosion wall. The natural way to match many patterns at once is a single combined automaton, and the documented failure mode of that approach is DFA state explosion: RE2::Set and Rust's `RegexSet` both report it past a few dozen patterns [3–5], and our own merged prototype hit it as a correctness failure (§3). Our response—one small DFA per pattern instead of one large shared one—is what keeps the lazy DFA usable at our pattern count, at the cost of giving up single-pass matching, which the selective merges then recover for the two subsets where it is exactly safe. Hyperscan [6] is the production multi-pattern matcher to position against; it is fast and SIMD-accelerated but x86-only, so it fails our portability constraint and is out of scope as a shippable option, not as prior art. The closest structural cousin is regex decomposition for database query evaluation [7], which shares the extract-a-literal-then-confirm shape; our always-candidate patterns (the pure-digit class with no extractable literal) are precisely the case that decomposition cannot help, and the reason we cannot rely on a literal filter alone.

What we deliberately do not cite as a basis. There is a line of work on automatic rule grouping and pattern partitioning—formulating the assignment of patterns to automata as a graph-partitioning problem and attacking it with heuristics—to minimize total DFA states. We share its vocabulary ("grouping") but not its method: our two merges are chosen by a syntactic test that recognizes two exactly-disjoint subsets the domain happens to contain, not by any partitioning algorithm over a conflict graph. Under our restricted pattern contract the general grouping problem degenerates into trivial structural recognition plus exact linear passes, and the honest framing is specialization, not a new grouping algorithm. We name this line only to disclaim it as a basis, rather than borrow citations for a technique we did not use.

9 CONCLUSION

The one idea to carry away is that engine replacement in production is a constrained search, not a benchmark: the fastest engine and the shippable engine are different objects, and the gap between them is set by the constraints—zero dependencies, exact output, thread-safety—not by raw speed. Treating those constraints as first-class inputs to the algorithm choice, rather than as deployment afterthoughts, is what turned an "embarrassingly slow C extension" into a tractable design problem with a clear answer.

Two of the findings along the way are reusable beyond this library. The shared-literal pre-filter, the standard reflex for multi-pattern matching, is not free: its benefit is bounded by the positions it can reject, so it erodes with match density and, in our configuration, loses to the very engine it fronts at every density we measured. And a multi-pattern lazy-DFA matcher can be made re-entrant in a managed runtime by partitioning state into shared-immutable and per-thread halves—*including the lazy cache the construction fills mid-scan*—which is the enabling step for releasing the interpreter lock safely.

The boundaries are equally concrete. The shippable engine leans on two specializations that hold because the domain is a closed, curated catalogue: a pure-digit group and a unique-prefix IBAN union. On a pattern set without such exactly-disjoint structure those merges simply do not fire, and the engine falls back to its per-pattern core—so the result is a tailored fit to a known catalogue, not a general-purpose regex engine, and we present it as such.

For a practitioner facing a similar replacement, the transferable method is the checklist this search became: measure across the input axis the use case cannot control, make exactness a gate rather than an aspiration, and decide eligibility before speed. The full prototype search, dead ends included, is the evidence that the order of those steps is what mattered.

ACKNOWLEDGMENTS

This paper, and the engineering it reports, were produced by the sole author working with Claude (Anthropic) as an instrument under the author's direction. The author set the hypotheses and the algorithmic approach, directed the research, benchmarking, and prototyping, and reviewed and is accountable for every claim and result. Appendix A gives the full account of how the tool was used and where the author's judgement overrode it.

DATA AND ARTIFACT AVAILABILITY

The data_redactor library, the prototypes, the benchmark and profiling harnesses, the raw results, and the figure/table generators are in the public project repository at https://github.com/danielefrisanco/data_redactor. A self-contained Docker artifact (`paper/repro/`) pins the toolchain of §6 and regenerates the density sweep, the Callgrind instruction-attribution table (Table 1), and the figures with a single command: `make -C paper/repro all`. Absolute timings depend on the host CPU; the qualitative results—the density crossover and the instruction attribution—are reproducible across machines, the latter exactly (it counts instructions, not cycles). The full sweep, including the slow glibc baseline, is regenerated by the artifact; an earlier power-saving draft of the baseline is retained in the data record for provenance (§7).

A HOW THIS WORK WAS PRODUCED

This work was carried out by the sole author with a large language model (Claude, Anthropic) used as an instrument throughout. Because the tool's involvement was substantial, this appendix records what it did, what the author did, and where the two diverged, so the reader can calibrate trust in the results rather than guess at it. The short version: the author owns the hypotheses, the engineering decisions, and accountability for every number; the tool accelerated research, implementation, measurement, and writing under that direction.

What the author set. The author chose the problem and its constraints (the three binary gates and the all-density robustness objective of §3), formed the central hypotheses—that a per-pattern lazy DFA could clear the pattern count where a single shared DFA could not, and that an extract-a-literal pre-filter pipeline would lose to a plain JIT at every density—and specified the algorithmic shape of the engine: a single left-to-right pass over the input that tests every pattern at each position,

rather than the original scheme of one full scan of the string per pattern; the two selective merges; and the intuition for why only the pure-digit and IBAN subsets are safe to merge. These were the author's design commitments, stated before the experiments that tested them.

What the tool did under direction. Given those hypotheses and designs, the tool was directed to survey related work and assemble candidate citations, to implement the prototype engines and the benchmark and profiling harnesses, to run the density sweep and the Callgrind attribution, and to draft and revise this manuscript. Each of these was a directed task with an author-specified goal, not an autonomous decision; the author read the harness code, the raw results, and the prose, and revised or rejected output that did not hold up.

Where the author's judgement overrode the tool. Two cases are worth recording because they show the division of labour working as intended. First, an early account of the incumbent's slowness attributed the cost largely to heap allocation. The author was unconvinced and directed a direct measurement: first a timing micro-benchmark isolating the per-call allocation, which already showed the malloc churn was only a few percent of runtime and not worth removing, and later a Callgrind instruction-attribution study that quantified it precisely. Both *disproved* the original account: allocation is under 1% of the incumbent's instructions, and the cost is dominated by automaton evaluation and per-call search setup (§5, Table 1). The paper reports the corrected, measured account, not the original inferred one. Second, the bibliography was assembled with several placeholder entries; the author verified each remaining citation against its primary record (DBLP, arXiv, or the publisher) before it was allowed to stand, and entries that could not be confirmed were left uncited rather than printed on trust.

Accountability. The author has reviewed the full manuscript, the experimental code, and the results, and is solely responsible for them. The tool is disclosed here as a method, not credited as an author: it holds no view, takes no responsibility, and made no claim that the author did not check and adopt as the author's own.

REFERENCES

[1] Alfred V. Aho and Margaret J. Corasick. 1975. Efficient String Matching: An Aid to Bibliographic Search. *Commun. ACM* 18, 6 (1975), 333–340. https://doi.org/10.1145/360825.360855

[2] Robert S. Boyer and J Strother Moore. 1977. A Fast String Searching Algorithm. *Commun. ACM* 20, 10 (1977), 762–772. https://doi.org/10.1145/359842.359859

[3] Russ Cox. 2007. Regular Expression Matching Can Be Simple and Fast (but is slow in Java, Perl, PHP, Python, Ruby, ...). https://swtch.com/~rsc/regexp/regexp1.html. Series incl. the lazy/hybrid DFA construction.

[4] Russ Cox et al. 2010. RE2: A Principled Approach to Regular Expression Matching. https://github.com/google/re2. RE2::Set: DFA state explosion past 30 patterns. Accessed 2026..

[5] Andrew Gallant et al. 2014. The Rust `regex` crate (`RegexSet`). https://docs.rs/regex. RegexSet: same state-explosion wall as RE2::Set. Accessed 2026..

[6] Xiang Wang, Yang Hong, Harry Chang, KyoungSoo Park, Geoff Langdale, Jiayu Hu, and Heqing Zhu. 2019. Hyperscan: A Fast Multi-pattern Regex Matcher for Modern CPUs. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 631–648. SIMD multi-pattern matcher; x86-only – disqualified for our portability.

[7] Ling Zhang, Shaleen Deep, Avrilia Floratou, Anja Gruenheid, Jignesh M. Patel, and Yiwen Zhu. 2023. Exploiting Structure in Regular Expression Queries. *Proceedings of the ACM on Management of Data (PACMMOD)* 1, 2, Article 152 (2023), 28 pages. https://doi.org/10.1145/3589297 System name: BLARE. Closest structural cousin: extract-a-literal + confirm decomposition.